Cortex

A Personal Second Brain Agent Network

An open-source, privacy-first multi-agent system that passively indexes your digital life and proactively surfaces connections, drafts decisions, and suggests actions — before you ask.

Version	1.0 (Draft)
Status	Proposed — Pre-Build
Last Updated	April 19, 2026
Document Owner	Product & Engineering
Target Release	Alpha: Q3 2026 · Public v1: Q1 2027

---

1. Executive Summary

Cortex is a personal, always-learning intelligence layer that sits across a user's entire digital surface area — notes, email, code repositories, bookmarks, chat, calendar — and turns fragmented context into proactive, actionable insight. It is architected as a multi-agent system with persistent cross-session memory and standardized connectors, running locally-first with optional cloud augmentation.

The thesis is simple: people already have a second brain. It just isn't connected. Notes live in Obsidian. Decisions live in Slack. Work artifacts live in GitHub and Notion. Commitments live in Gmail. No single tool has all the context, so no tool can reason across it. Cortex closes that loop by indexing everything into a unified semantic memory and letting specialized agents — Researcher, Planner, Drafter, Connector — operate on that memory continuously.

Why this is buildable now, in weeks rather than months: **MCP (Model Context Protocol)** provides standardized connectors to Gmail, GitHub, Notion, Obsidian, Slack, Google Drive, and dozens more — eliminating the bespoke integration work that previously dominated timelines. Mature embedded vector databases with first-class Rust APIs (LanceDB, Qdrant) make persistent memory trivial. The Rust AI ecosystem has matured dramatically: **rig** and **swiftide** handle agent orchestration and RAG pipelines, **candle** and **mistral.rs** run local inference, and the official MCP Rust SDK makes connector work native. The hard parts are no longer infrastructure; they are product design and trust.

Cortex is built end-to-end in Rust. This is a deliberate bet: the product must run continuously in the background on the user's machine with low CPU, low memory, and no GC pauses; it must ship as a small, signed, single binary that non-trivial users will trust with their email and source code; and it must hold a hard privacy posture where every dependency is auditable. Rust is the only mainstream choice that satisfies all three. It also collapses the deployment story — no Python environments, no venv drift, no "why is the daemon using 800 MB of RAM" support tickets.

The market opportunity is meaningful but narrow: many teams are building "AI agents" and many are building "second brain" tools, but none have shipped a clean, open-source, privacy-respecting integration of the two. Cortex aims to be that reference implementation — and, through open distribution, the category-defining project.

---

2. Problem Statement

2.1 The Fragmentation Problem

Knowledge workers spend, by various industry estimates, between 20% and 30% of their working hours searching for information they already have. The cost is not just lost time — it is lost reasoning. A decision made in a Slack thread three weeks ago cannot inform today's PR review because the two systems do not talk. A note taken during a customer call cannot surface during the next sprint planning because it lives in a separate silo.

2.2 The Reactive AI Problem

Current AI assistants are almost entirely reactive: the user formulates a question, the assistant answers. This places the entire cognitive load of knowing what to ask on the user. The user must remember a connection exists before they can query for it. This is backwards. Valuable insight most often comes from connections the user *did not know existed* — which by definition they cannot ask about.

2.3 The Integration Tax

Previous attempts at cross-system personal intelligence required months of bespoke integration work per data source, constant breakage as APIs shifted, and maintenance burdens that exceeded the value delivered. The emergence of MCP as a de facto standard, combined with production-grade local vector stores, collapses this timeline by roughly an order of magnitude.

2.4 The Trust Problem

The most valuable personal AI is also the most invasive. A system that reads your email, your code, and your private notes must be architected for trust from the first line of code — not retrofitted. Existing closed-source offerings ask users to send their entire digital life to a third-

party cloud with opaque retention policies. This is a non-starter for a meaningful portion of the target audience.

---

3. Goals and Non-Goals

3.1 Goals

- Build a local-first, multi-agent personal intelligence system that indexes the user's primary digital surfaces (email, notes, code, calendar, chat, files) into a unified semantic memory.
- Enable proactive surfacing — Cortex should suggest, draft, and connect without being explicitly prompted for each action.
- Ship a reference open-source implementation that establishes the architectural pattern and becomes the default starting point for this category.
- Achieve a credible trust posture: local-first, auditable, with clear data boundaries and user-controlled retention.
- Reach a time-to-first-value under 15 minutes for a technically capable user.

3.2 Non-Goals

- Cortex is not a consumer chatbot. It is infrastructure with a UI layer; casual users are not the initial target.
- Cortex is not a replacement for Obsidian, Notion, or Gmail. It augments them; it does not seek to own the source data.
- Cortex will not ship a proprietary model. It is orchestration and memory over best-available models (Claude, local Llama variants, etc.).
- Cortex will not offer a hosted multi-tenant SaaS in v1. A managed option may come later; the architecture must remain single-tenant and self-hostable.
- Cortex will not handle real-time collaboration or team-shared brains in v1. The initial product is explicitly single-user.

---

4. Target Users and Use Cases

4.1 Primary Persona — The Technical Polymath

Senior engineers, founders, researchers, and independent consultants who operate across many contexts simultaneously. They already use power tools (Obsidian, GitHub, a terminal, a local LLM setup), they are comfortable running a Docker container, and they care about data

ownership. They are the people most likely to have tried to build a version of this themselves and given up halfway through the connector layer.

4.2 Secondary Persona — The Engineering Leader

Staff-plus engineers and engineering managers who juggle code review, architecture decisions, 1:1 notes, and strategy documents. Their pain is specifically the handoff between what was decided in a meeting and what ends up in a PR. Cortex targets that seam.

4.3 Representative Use Cases

Use Case	Description
Weekly review synthesis	Cortex composes a weekly digest from commits, email threads, calendar events, and notes — highlighting themes, unresolved decisions, and commitments the user made that are still open.
Proactive connection surfacing	While the user is drafting a design doc in Obsidian, Cortex surfaces a six-month-old note, a relevant GitHub issue, and a customer email thread — without being asked.
Draft-before-you-ask	Cortex notices an email arriving that requires a decision it has context on, and prepares a draft reply sitting in a review queue for the user to approve, edit, or discard.
Cross-session memory for coding	A coding agent operating against the repo already knows the architectural decisions logged in the user's notes, reducing repetitive context-setting.
Commitment tracking	Cortex extracts commitments ("I'll send this by Thursday") from outbound channels and tracks their status without requiring the user to explicitly log them.

---

5. Product Overview

5.1 Conceptual Model

Cortex is organized around three layers. The **ingestion layer** passively pulls and normalizes data from connected sources via MCP servers. The **memory layer** stores that data as embeddings in a local vector database alongside structured metadata, with entity resolution and temporal tracking. The **agent layer** is a small set of specialized agents coordinated by an orchestrator that can plan, reason, and execute across memory and back into user-facing channels.

5.2 The Agents

Agent	Responsibility
Indexer	Continuously ingests from connected sources. Normalizes, chunks, embeds, and writes to memory. Runs on a low-priority background schedule.
Researcher	Answers direct queries by retrieving across memory, ranking, and synthesizing. Handles citations back to source.
Connector	Runs on a cadence or trigger; looks for non-obvious semantic links between recent activity and older memory. Produces insight cards.
Drafter	Generates draft replies, summaries, and documents when a trigger fires — new email requiring response, meeting ending with notes, PR awaiting review.
Planner	Breaks user goals into actions, sequences them, and hands execution to the appropriate agent or surfaces the plan for user review.
Orchestrator	The top-level controller. Routes requests, maintains session state, enforces policy (what the system is allowed to read, write, and act upon).

5.3 Proactive vs. Reactive Modes

Cortex operates in both modes simultaneously. Reactive mode is the familiar chat interface — a query in, an answer out, with citations. Proactive mode runs continuously in the background, producing a stream of *insight cards* that the user reviews at their cadence (morning briefing, end-of-day, notification-driven). A card is discardable, actionable, and traceable to its source data.

---

6. Functional Requirements

6.1 Ingestion & Connectors (v1)

The following MCP-compatible sources must be supported at v1. Each must handle initial backfill, incremental sync, deletion propagation, and rate-limit-aware retry.

- Gmail — messages, threads, labels, attachments metadata.
- GitHub — repositories, issues, PRs, commit messages, code (opt-in per repo).
- Obsidian — vault contents, link graph, tags.
- Notion — pages, databases, comments.

- Google Calendar — events, attendees, notes.
- Google Drive — document text extraction (Docs, Sheets, Slides, PDFs).
- Slack — selected channels and DMs (explicit opt-in per workspace and channel).
- Local filesystem — a user-designated folder for PDFs, markdown, etc.
- Browser bookmarks and read-it-later (Pocket/Readwise/Matter).

6.2 Memory

- Local-first vector database (LanceDB recommended for v1; Chroma and Qdrant as pluggable backends).
- Hybrid retrieval: dense embedding search + BM25 keyword + metadata filters, reranked.
- Temporal awareness: every memory has a source timestamp and an ingestion timestamp; retrieval can be time-filtered ("what did I think about X last month?").
- Entity resolution: people, projects, repos, and documents are resolved to canonical entities across sources.
- Memory decay and summarization: older, rarely-accessed chunks are summarized into higher-level memories to keep retrieval signal-to-noise high.
- Deletion honoring: when a source item is deleted upstream, its memory representation is removed on next sync.

6.3 Agents & Orchestration

- Agent framework built on `rig` (Rust) for typed, composable agent pipelines with first-class tool-calling and provider abstraction.
- Orchestration is modeled as explicit state machines using a tokio-based workflow layer (petri-net or `statig` crate) — preferred over freeform autonomy for traceability.
- Each agent run produces a structured trace — inputs, tool calls, retrieved memory IDs, final output — serialized to SQLite and inspectable in the UI.
- Model flexibility: Claude (primary) via async HTTP client, local models via `mistral.rs` or `candle` embedded directly in-process, and OpenAI-compatible endpoints as a fallback. Model choice is configurable per agent.
- Deterministic routing: the orchestrator decides which agent handles a request based on declared capabilities, not freeform delegation.

6.4 Proactive Surface

- Insight cards delivered via: (a) a native desktop app panel, (b) a daily email digest, and (c) an MCP endpoint so other tools can consume them.

- Every card includes: a claim, the memories it draws from, a suggested action (if any), and dismiss/save/act controls.
- User feedback on cards (useful / not useful / already knew) tunes future surfacing. This is the primary learning loop.
- Quiet hours and rate limits on card production — proactive must never become noisy.

6.5 User Interfaces

- Desktop app built on Tauri v2 with a Rust backend exposing a typed command surface to a React/TypeScript frontend. The daemon and the UI are the same binary — no IPC across language boundaries.
- CLI (a separate thin binary sharing the core crate) for power users — query, ingest, status, and trace inspection.
- Cortex itself exposes an MCP server over stdio and HTTP, so Claude Code, Cursor, Zed, and other MCP-aware tools can query personal memory during coding sessions.

6.6 Actions & Writebacks

Any action that writes to an external system (send email, create PR comment, write to Notion) is gated behind explicit user approval in v1. The system may *draft* autonomously; it may not *send*. A later version may allow scoped autonomous actions per user-configured policy.

---

7. Non-Functional Requirements

Attribute	Requirement
Privacy	Local-first storage by default. No data leaves the user's machine except to LLM inference endpoints, and those can be fully local (Ollama) if desired. All inference calls, inbound and outbound, are logged and inspectable.
Security	OAuth tokens stored in OS keychain. Filesystem data at rest is encrypted. Network access for connectors follows least-privilege scopes.
Performance	Cold retrieval over 100k memories: median under 150 ms. Proactive agent loop should not exceed 2% sustained CPU and 250 MB resident memory on a modern laptop. Cold-start of the daemon under 300 ms.
Reliability	Connector failures must be isolated — one broken sync does not break others. Ingestion is idempotent and resumable. Rust's type system is leaned on heavily: panics in the daemon are treated as bugs, not runtime conditions.

Attribute	Requirement
Observability	Every agent run is traceable. Users can see, for any card or answer, the exact retrieved memories and tool calls that produced it.
Portability	Memory and configuration are exportable as a single directory the user can back up, move, or delete. No vendor lock-in.
Extensibility	New connectors are MCP-standard and can be added without touching Cortex core.

8. Technical Architecture

8.1 Recommended Stack

Cortex is a single-language codebase. Every component — the background daemon, the agent runtime, the connector layer, the embedding pipeline, the UI backend, and the CLI — is Rust. This is a design constraint, not a preference. A personal-intelligence daemon that runs 24/7 on a user's laptop cannot afford the memory profile of a Python process tree or the cold-start latency of a Node runtime, and the trust posture demands a small auditable dependency graph and a signed single binary.

Layer	Choice	Rationale
Language & runtime	Rust (stable), `tokio`	Single-language codebase. Tokio is the async runtime across the daemon, connectors, and agent orchestration.
Agent framework	`rig`	Typed, composable agent pipelines with first-class tool-calling and provider abstraction. The most mature Rust agent library as of 2026.
RAG & pipelines	`swiftide`	Ingestion and retrieval pipelines with streaming, transformers, and embedding adapters. Complements `rig` for the indexing side.
Memory store	LanceDB (primary), Qdrant (pluggable)	LanceDB ships a native Rust API with columnar storage, strong metadata filters, and embedded mode — ideal for local-first. Qdrant available for users who want a richer filter DSL.
Structured metadata	SQLite via `sqlx`	Entity tables, audit log, agent traces, card feedback. Embedded, zero-ops, backup is a file copy.
Embeddings	`fastembed-rs` (default), `candle` (GPU), API-based (optional)	`fastembed-rs` runs BGE and Nomic models natively with ONNX Runtime. Candle offers GPU paths. API embeddings are available but off by default to preserve the privacy posture.
Local inference	`mistral.rs` or `candle`	In-process local inference for users who want zero data leaving the machine. No Ollama sidecar required, though Ollama remains supported as an external endpoint.
Remote inference	Claude API (primary), any OpenAI-compatible endpoint	Async HTTP via `reqwest`. Per-agent model configuration. Cheap, fast models for routing; stronger models for synthesis.
MCP client & server	Official MCP Rust SDK (`rmcp`)	Cortex consumes upstream MCP servers as a native client, and exposes itself as an MCP server so other tools can query personal memory.
App shell	Tauri v2	Rust backend, React/TypeScript frontend. Small signed binary, native performance, cross-platform.
Background daemon	systemd / launchd / Windows Service via a	The daemon, the UI backend, and the CLI all link the same core crate. No cross-language IPC.

Layer	Choice	Rationale
	single Rust service crate	
Observability	`tracing` + `tracing-subscriber`	Structured spans for every agent run, connector sync, and inference call. Exportable to a local file or OTLP endpoint.

8.2 Workspace Layout

The repository is a Cargo workspace. A small set of well-scoped crates keeps compile times manageable and makes the trust boundary explicit — a security reviewer can read the core crate without reading the UI.

- `cortex-core` — domain types, memory traits, policy engine. No I/O. No async. Pure logic, fully testable.
- `cortex-memory` — LanceDB/Qdrant adapters, SQLite metadata, entity resolution, temporal indexing.
- `cortex-agents` — `rig`-based agent definitions, orchestrator state machine, trace emission.
- `cortex-connectors` — MCP client wrappers per source (Gmail, GitHub, Notion, Obsidian, Slack, Drive, Calendar, FS). Each is its own crate behind a uniform trait.
- `cortex-inference` — provider abstraction over Claude, OpenAI-compatible, `mistral.rs`, `candle`.
- `cortex-daemon` — the always-on service. Schedules ingestion, runs the Connector agent loop, handles IPC.
- `cortex-app` — Tauri binary. React frontend under `src-ui/`, Rust commands under `src-tauri/`.
- `cortex-cli` — `clap`-based CLI, shares the core crate with the daemon and app.

8.3 Data Flow

1. MCP connectors poll or stream from source systems. New or changed items land in an ingestion queue.
2. The Indexer agent pulls from the queue, normalizes each item into a canonical document shape, extracts entities, chunks, embeds, and writes to LanceDB with metadata.
3. The Connector agent runs on a cadence, executing template queries that look for novel semantic links between recent items and the broader memory. Results become candidate insight cards.

4. Candidate cards pass through a relevance filter (user feedback history, novelty check, policy) before being surfaced.
5. User queries flow through the Orchestrator, which routes to Researcher or Drafter. Retrieved memories and full reasoning traces are returned alongside the answer.
6. Actions — drafts, writebacks, scheduled tasks — are queued for user approval unless pre-authorized by policy.

8.4 Trust Architecture

- Local-first: the default install has no cloud dependency beyond the LLM inference endpoint the user chooses.
- Pluggable inference: the user can point inference at a fully local model; in that configuration, no data leaves the machine.
- Per-source opt-in: each connector is off by default. Turning one on requires explicit OAuth consent and scope selection.
- Audit log: every read from a source and every inference call is logged locally with timestamp, purpose, and returned size.
- Deletion primitive: a single command purges all memory derived from a given source or entity, with verification.

---

9. Milestones and Release Plan

Milestone	Timeline	Scope
M0 — Skeleton	Weeks 1-2	Cargo workspace with core crates stubbed. Tauri v2 shell. `rig`-based orchestrator stub. LanceDB + SQLite wired up. Claude as default inference provider. End-to-end "hello from memory" roundtrip.
M1 — First Connectors	Weeks 3-5	Gmail, GitHub, and local filesystem connectors via MCP. Indexer agent. Basic chat interface with retrieval.
M2 — Second Wave	Weeks 6-8	Obsidian, Notion, Calendar connectors. Entity resolution. Reranked hybrid retrieval.
M3 — Proactive Alpha	Weeks 9-12	Connector agent and insight card feed. Daily digest email. Feedback loop. Closed alpha with 25-50 users.

Milestone	Timeline	Scope
M4 — Drafter & Actions	Weeks 13-16	Draft-reply and draft-summary flows. Approval UI. Action audit log.
M5 — Public Beta	Weeks 17-20	Polish, onboarding under 15 minutes, documentation, open-source launch with reference deployments.
M6 — v1.0	Weeks 21-26	Stability hardening, MCP-self-server, initial community connector ecosystem, v1.0 release.

---

10. Success Metrics

10.1 North Star

Weekly active sessions per user, weighted by *acted-upon insight cards*. The metric rewards both engagement and demonstrated value — a user who receives 50 cards and acts on zero is a failure; a user who receives 10 and acts on 4 is a success.

10.2 Leading Indicators

- Time-to-first-value: median time from install to first useful card. Target: under 30 minutes at public beta, under 15 minutes at v1.
- Card usefulness rate: fraction of surfaced cards marked useful (or acted upon without dismissal). Target: above 40% by M5.
- Connectors per active user: median number of active connectors after week 2 of use. Target: at least 4.
- Retention: week-4 retention of users who complete onboarding. Target: above 50% by v1.

10.3 Ecosystem Metrics

- GitHub stars, forks, and external contributions — leading indicators of category-defining status.
- Third-party MCP connectors targeting Cortex specifically.
- Mentions in the technical press and in adjacent projects' documentation as a reference architecture.

---

11. Risks and Mitigations

Risk	Severity	Mitigation
The proactive surface becomes noise and users disable it	High impact; medium likelihood	Start conservative with rate limits. Card feedback loop is built in from day one. Track usefulness rate as a primary KPI and halt new card types that fall below threshold.
MCP standard fragments or a competing standard emerges	Medium impact; medium likelihood	Architect the connector layer behind an internal interface so the protocol is swappable. Contribute upstream to keep MCP healthy.
Inference costs make heavy users expensive to serve	Medium impact; low likelihood in local-first model	Local-first architecture pushes inference cost to the user by design. Offer a hosted option only when unit economics are proven.
Connector maintenance burden outpaces core team capacity	High impact; high likelihood over time	Invest early in connector conformance tests. Encourage community ownership. Prioritize depth over breadth — fewer, better connectors.
Privacy or security incident in a well-meaning beta	Very high impact; low but non-zero likelihood	Local-first reduces blast radius. Ship with an audit log, a kill switch, and a documented incident response process. External security review before public beta.
A large incumbent ships a competitive closed-source product	Medium impact; high likelihood	The advantage is openness, privacy posture, and extensibility. Compete on what a closed-source vendor cannot replicate: self-hosting and an ecosystem of user-authored connectors.
Agent loops produce low-quality outputs that erode trust	High impact; medium likelihood	Keep agents narrow and deterministic. Require traceability for every output. Invest in eval harnesses for each agent from M1 forward.
Rust AI ecosystem lags the Python ecosystem for a specific capability	Medium impact; medium likelihood	The provider abstraction in `cortex-inference` is a thin trait. For capabilities not yet available in Rust, a sidecar process is acceptable as an escape hatch — but it is a failure mode, not a design pattern. Contribute upstream to `rig`, `swiftide`, and `candle` where gaps appear.
Hiring bottleneck — Rust + agents is a	Medium impact; medium	Keep the core team small and senior. Lean on the open-source community for connectors (the highest-volume,

Risk	Severity	Mitigation
narrow talent intersection	likelihood	most-standardized work). Document the architecture well enough that a strong Rust engineer unfamiliar with the agent space can contribute within a week.

---

12. Open Questions

- Should v1 include any form of encrypted peer-to-peer sharing, or is single-user strictly enforced until v2?
- What is the right default cadence for the Connector agent — continuous, hourly, daily digest only?
- Do we ship a hosted inference proxy (for users who want the product but don't have API keys), and what is the business model if so?
- Should the Drafter be allowed to take low-risk autonomous actions (e.g., triaging labels, archiving) under policy, or is human approval always required?
- Is there a freemium path (OSS core + paid managed) that does not compromise the trust posture?
- How aggressive should memory decay and summarization be — and should it be user-configurable, or a product opinion?

---

13. Appendix

13.1 Glossary

- **MCP (Model Context Protocol):** an open protocol, introduced by Anthropic, for connecting LLM applications to external data sources and tools through a standardized server interface.
- **Insight card:** the atomic unit of proactive surfacing in Cortex — a short, dismissible, traceable artifact with an optional suggested action.
- **Hybrid retrieval:** a retrieval strategy combining dense (embedding-based) and sparse (keyword-based) search, typically with a final reranking pass.
- **Entity resolution:** the process of recognizing that, e.g., "Jane Smith" in Gmail, "jsmith" in GitHub, and "Jane" in a calendar event refer to the same person.

13.2 Related Prior Art

- Rewind, Reflect, Mem.ai — commercial closed-source entries in adjacent space.
- Khoj, PrivateGPT — open-source predecessors focused narrowly on local retrieval, without the agent layer.
- LangChain/LangGraph agent examples — orchestration building blocks, not products.
- Obsidian plugins (Smart Connections, Copilot) — valuable but single-source.

13.3 Design Principles

1. Local-first, always. Cloud is a convenience, never a requirement.
2. Traceable over clever. Every output must be explainable by pointing at its source memories.
3. Draft, do not send. Autonomy is earned, not granted by default.
4. Depth over breadth. Ten excellent connectors beat fifty mediocre ones.
5. The user owns the data, the schema, and the exit.
6. One language, one binary. Rust end-to-end; no cross-language IPC; no runtime to install.